Лабораторна робота № 5

Географічні дані OpenStreetMap: від тегів до маршруту

Інтелектуальний аналіз та візуалізація даних · 4 години · 4 бали

Постановка

Ви — аналітик у туристичному застосунку. Завдання: для заданого міста **відібрати десять найвартісніших історико-культурних об'єктів** і **спланувати піший маршрут**, який починається на залізничному вокзалі, обходить усі десять і повертається на вокзал.

Слово «найвартісніші» в даних не записане. Критерій ви побудуєте самі — і захистите його.

Потім те саме для **тридцяти** об'єктів. Результат цієї, більшої задачі стане базою порівняння в **лабораторній роботі № 8**, де ту саму задачу розв'язуватимуть генетичним та мурашиним алгоритмами.

Що здаєте

1. Цей ноутбук із виконаними комірками (усі виводи збережені).
2. Файл `lr5_результати.json` — результати для ЛР8.
3. Карту `lr5_маршрут.html`.
4. Звіт у кінці ноутбука: відповіді на сім питань.

Що оцінюється

критерій	0	2	3	4
Відбір об'єктів	теги взято навмання	фільтр працює	фільтр обґрунтовано	показано, що відкинуто і чому
Критерій рейтингу	немає	є формула	ваги пояснено	порівняно з альтернативним критерієм
Маршрут	не будується	будується	перевірено коректність	пояснено різницю методів числом
Карта	немає	точки на карті	маршрут по ребрах графа	видно різницю з прямими відрізками

критерій	0	2	3	4
Звіт	немає	формально	з числами	з висновком, якого не було в завданні

0. Ваш варіант

Впишіть прізвище та номер у списку групи. Місто підтягнеться само.

In [1]:
```
STUDENT_NAME = 'Кузнєцов Макар'
LIST_NUMBER  = 8      # номер у списку групи

CITIES = [
    'Львів', 'Київ', 'Харків', 'Одеса', 'Дніпро',
    'Чернівці', 'Івано-Франківськ', 'Тернопіль', 'Ужгород', 'Луцьк',
    'Рівне', 'Хмельницький', 'Вінниця', 'Житомир', 'Чернігів',
    'Черкаси', 'Полтава', 'Суми', 'Кропивницький', 'Запоріжжя',
    'Миколаїв', 'Кривий Ріг', 'Кам\u2019янець-Подільський', 'Мукачево',
    'Коломия', 'Дрогобич', 'Біла Церква', 'Ніжин', 'Остріг', 'Умань',
]

assert STUDENT_NAME and LIST_NUMBER > 0, 'впишіть прізвище та номер у списку
CITY = CITIES[(LIST_NUMBER - 1) % len(CITIES)] + ', Україна'
print('варіант %d: %s' % (LIST_NUMBER, CITY))
```

```
варіант 8: Тернопіль, Україна
```

> Якщо викладач призначив вам інше місто — просто перепишіть `CITY` вручну і зазначте це у звіті.

In [2]:
```
import importlib.util, subprocess, sys
REQUIRED = ['osmnx', 'geopandas', 'shapely', 'pyproj', 'pyarrow',
            'folium', 'mapclassify', 'h3', 'networkx']
missing = [p for p in REQUIRED if importlib.util.find_spec(p) is None]
if missing:
    print('встановлюю:', ', '.join(missing), '...')
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q', *mi
print('усі пакети на місці')
```

```
усі пакети на місці
```

In [3]:
```
import warnings; warnings.filterwarnings('ignore')
import json, time, os, collections
import numpy as np
import pandas as pd
import geopandas as gpd
import networkx as nx
import matplotlib.pyplot as plt
import folium
from xyzservices import providers as xyz
```

```
BASEMAP = xyz.Esri.WorldGrayCanvas   # CARTO з 2026 р. вимагає ключ API
pd.set_option('display.width', 200)
print('geopandas', gpd.__version__)
```

```
geopandas 1.2.0
```

---

1. Два способи дістати дані

Робоча програма вимагає порівняти два способи доступу **за обсягом переданих даних і за часом**:

- **живий запит** — `osmnx` питає Overpass API просто зараз;
- **знімок** — ті самі дані, збережені у файли GeoParquet, читаються з диска.

Нижче — живий запит. Запустіть його **один раз**: він збереже теку зі знімком, і всі наступні запуски читатимуть з неї.

> Overpass — безкоштовний сервіс і обмежує частоту. Якщо запит не пройшов — зачекайте хвилину й повторіть. Якщо не проходить взагалі, візьміть резервний знімок у викладача і вкажіть це у звіті.

In [4]:
```
import osmnx as ox

OUT = 'osm_' + CITY.split(',')[0].strip().lower().replace(' ', '_')
os.makedirs(OUT, exist_ok=True)
ox.settings.use_cache = True
ox.settings.cache_folder = os.path.join(OUT, 'cache')
ox.settings.requests_timeout = 300
ox.settings.overpass_rate_limit = True

TOURISM_OK = ['museum', 'attraction', 'gallery', 'artwork', 'viewpoint']

def cache_bytes():
    import pathlib
    p = pathlib.Path(ox.settings.cache_folder)
    return sum(f.stat().st_size for f in p.glob('*.json')) if p.exists() els

print('тека знімка:', OUT)
```

```
тека знімка: osm_тернопіль
```

Дві налаштовані дрібниці, без яких запит не пройде

Це не формальності — кожна з них ламає збір по-своєму, і жодна не схожа на свою справжню причину.

Представитись. Overpass відповідає **HTTP 406** клієнтові, який прийшов із типовим заголовком `python-requests`. `osmnx` надсилає свій `User-Agent` сам, але якщо ви звертаєтесь до сервісу напряму — надсилайте теж. Рядок має бути **лише**

латиницею: HTTP-заголовки передаються в latin-1, і кирилиця кладе запит помилкою `UnicodeEncodeError` ще до відправлення.

Обрати машину. За іменем `overpass-api.de` стоїть кілька серверів, і частина з них відмовляє у з'єднанні. `osmnx` закріплює одну адресу на всю сесію — навмисно, щоб не порушувати облік слотів. Якщо закріпиться та, що відмовляє, впаде весь збір. Тому адресу обираємо самі, вже перевірену.

In [5]:
```
import socket

USER_AGENT = ('LvivPolytechnic-SK26-coursework/1.0 '
              '(https://lpnu.ua; student lab assignment)')
assert USER_AGENT.isascii(), 'HTTP-заголовки — latin-1, кирилиця тут не прац
ox.settings.http_user_agent = USER_AGENT


def pin_working_ip(hostname, port=443, timeout=8):
    """Закріпити для цього імені адресу, яка справді відповідає."""
    try:
        infos = socket.getaddrinfo(hostname, port, socket.AF_INET, socket.SO
    except Exception as e:
        print('не вдалось резолвити %s: %s' % (hostname, type(e).__name__))
        return None
    ips = sorted({i[4][0] for i in infos})
    good = []
    for ip in ips:
        s = socket.socket(); s.settimeout(timeout)
        try:
            s.connect((ip, port)); good.append(ip)
        except Exception:
            pass
        finally:
            s.close()
    print('%s: машин %d, відповідають %d %s' % (hostname, len(ips), len(gooc
    if not good:
        return None
    original = socket.gethostbyname
    socket.gethostbyname = lambda h: good[0] if h == hostname else original(
    print('закріплено', good[0])
    return good[0]


pin_working_ip('overpass-api.de')
```

```
overpass-api.de: машин 2, відповідають 2 ['162.55.144.139', '65.109.112.52']
закріплено 162.55.144.139
```

Out[5]:
```
'162.55.144.139'
```

> Якщо жодна машина не відповіла — виконайте комірку ще раз за хвилину, або візьміть у викладача резервний знімок і зазначте це у звіті. Інші дзеркала перелічені на сторінці Overpass API; адресу можна підставити у `ox.settings.overpass_url`.

Завдання 1.1

Допишіть функцію `fetch`: вона має зберегти шар у `.parquet`, а якщо файл уже є — прочитати його, **не звертаючись до мережі**. Поверніть також, скільки МБ пішло мережею і скільки секунд це зайняло.

Підказка щодо `clean`: у OSM одна колонка може містити і число, і список (ребро, склеєне з кількох шляхів). Parquet вимагає один тип на колонку, тому неоднорідну колонку треба звести до тексту **цілком**.

In [6]:
```
def clean(gdf):
    g = gdf.reset_index()
    for c in g.columns:
        if c == 'geometry':
            continue
        boxed = g[c].map(lambda v: isinstance(v, (list, dict, set, tuple))).
        if boxed or g[c].dtype == object:
            g[c] = g[c].map(
                lambda v: None if v is None or (isinstance(v, float) and pd.
                else (json.dumps(v, ensure_ascii=False)
                      if isinstance(v, (list, dict, set, tuple)) else str(v)
            ).astype('string')
    return g


COST = {}

def fetch(name, getter):
    """Повертає шар. Якщо файл уже збережено — читає з диска."""
    path = os.path.join(OUT, name + '.parquet')
    if os.path.exists(path) and os.path.getsize(path) > 0:
        t0 = time.time()
        g = gpd.read_parquet(path)
        COST[name] = {'спосіб': 'знімок', 'записів': len(g),
                      'передано_МБ': 0.0,
                      'файл_МБ': round(os.path.getsize(path) / 1e6, 3),
                      'секунд': round(time.time() - t0, 3)}
        return g

    t0, b0 = time.time(), cache_bytes()
    g = clean(getter())
    g.to_parquet(path, index=False)
    g = gpd.read_parquet(path)
    COST[name] = {'спосіб': 'живий запит', 'записів': len(g),
                  'передано_МБ': round((cache_bytes() - b0) / 1e6, 3),
                  'файл_МБ': round(os.path.getsize(path) / 1e6, 3),
                  'секунд': round(time.time() - t0, 1)}
    return g
```

In [7]:
```
boundary = fetch('boundary', lambda: ox.geocode_to_gdf(CITY))
poly = boundary.geometry.iloc[0]
```

```
print('межа міста отримана, площа %.1f км²'
      % (boundary.to_crs(32635).area.iloc[0] / 1e6))
```

```
межа міста отримана, площа 58.6 км²
```

In [8]:
```
heritage = fetch('heritage', lambda: ox.features_from_polygon(
    poly, {'historic': True, 'tourism': TOURISM_OK}))
stations = fetch('stations', lambda: ox.features_from_polygon(
    poly, {'railway': ['station', 'halt'], 'building': ['train_station']}))
amenities = fetch('amenities', lambda: ox.features_from_polygon(
    poly, {'amenity': ['cafe', 'restaurant', 'drinking_water', 'toilets']}))

# Мережа приходить двома таблицями одразу, тому їй окрема гілка.
NODES_P = os.path.join(OUT, 'road_nodes.parquet')
EDGES_P = os.path.join(OUT, 'road_edges.parquet')

if os.path.exists(NODES_P) and os.path.exists(EDGES_P):
    t0 = time.time()
    nodes, edges = gpd.read_parquet(NODES_P), gpd.read_parquet(EDGES_P)
    COST['мережа walk'] = {'спосіб': 'знімок', 'записів': len(nodes) + len(e
                           'передано_МБ': 0.0,
                           'файл_МБ': round((os.path.getsize(NODES_P)
                                             + os.path.getsize(EDGES_P)) / 1
                           'секунд': round(time.time() - t0, 3)}
else:
    t0, b0 = time.time(), cache_bytes()
    G_raw = ox.graph_from_polygon(poly, network_type='walk')
    nodes, edges = ox.convert.graph_to_gdfs(G_raw)
    clean(nodes).to_parquet(NODES_P, index=False)
    clean(edges).to_parquet(EDGES_P, index=False)
    nodes, edges = gpd.read_parquet(NODES_P), gpd.read_parquet(EDGES_P)
    COST['мережа walk'] = {'спосіб': 'живий запит', 'записів': len(nodes) +
                           'передано_МБ': round((cache_bytes() - b0) / 1e6,
                           'файл_МБ': round((os.path.getsize(NODES_P)
                                             + os.path.getsize(EDGES_P)) / 1
                           'секунд': round(time.time() - t0, 1)}

print('вузлів %d | ребер %d' % (len(nodes), len(edges)))
```

```
вузлів 14314 | ребер 38950
```

Завдання 1.2

Зведіть `COST` у таблицю і дайте відповідь у звіті: **у скільки разів читання знімка швидше за живий запит** і **скільки мегабайтів ви не передали мережею** при повторному запуску.

Контрольна ознака правильності: при **другому** запуску ноутбука всі рядки мають мати спосіб «знімок», а `передано_МБ` — нуль.

In [9]:
```
display(pd.DataFrame(COST).T)
```

	спосіб	записів	передано_МБ	файл_МБ	секунд
boundary	знімок	1	0.0	0.022	0.215
heritage	знімок	266	0.0	0.097	0.017
stations	знімок	6	0.0	0.036	0.014
amenities	знімок	433	0.0	0.095	0.017
мережа walk	знімок	53264	0.0	2.744	0.142

> **Чому пішохідна мережа, а не автомобільна.** У старому місті музеї стоять за сотню метрів один від одного, а автомобільних вузлів там одиниці. Кілька пам'яток сядуть на **один** вузол графа, і матриця віддалей скаже, що між ними нуль метрів. Задача розв'яжеться, відповідь буде неправильною, і повідомлення про помилку не з'явиться. У кроці 4 ви це перевірите явно.

---

2. Хто претендує на звання пам'ятки

У OpenStreetMap немає тега «визначна пам'ятка». Є два сімейства:

- `historic=*` — `castle`, `church`, `monument`, `memorial`, `ruins`, `tower`, `yes` …
- `tourism=*` — `museum`, `gallery`, `attraction`, `artwork`, `viewpoint`

Перетин неповний: музей у сучасній будівлі має лише `tourism`, руїни фортеці — лише `historic`. Тому беремо **об'єднання**.

📖 https://wiki.openstreetmap.org/wiki/Map_Features

Завдання 2.1

Відберіть кандидатів і **назвіть у звіті, скільки об'єктів ви відкинули й чому**.

In [10]:
```
mask = heritage['historic'].notna() | heritage['tourism'].isin(TOURISM_OK)
has_name = heritage['name'].notna()
cand = heritage[mask & has_name]
cand = cand.reset_index(drop=True)

print('усього у шарі heritage :', len(heritage))
print('підходять за тегами    :', int(mask.sum()))
print('з них мають name       :', int((mask & has_name).sum()))
```

```
усього у шарі heritage : 266
підходять за тегами    : 266
з них мають name       : 218
```

In [11]:
```python
assert 'cand' in dir(), 'спершу виконайте завдання 2.1'
print('=== historic ===');  print(cand['historic'].value_counts().head(10).t
print()
print('=== tourism ===');   print(cand['tourism'].value_counts().head(10).to
print()
print('типи геометрії:', cand.geometry.geom_type.value_counts().to_dict())
```

```
=== historic ===
historic
memorial    152
tomb          1
monument      1
stone         1
castle        1

=== tourism ===
tourism
artwork       40
attraction    23
museum         8
viewpoint      1

типи геометрії: {'Point': 205, 'Polygon': 13}
```

> **Перевірте себе.** Серед типів геометрії мають бути і `Point`, і `Polygon`. Якщо самі лише полігони — ви відфільтрували зайве: пам'ятники, меморіали та оглядові точки в OSM є **вузлами**, у них немає будівлі.

Якщо кандидатів менше тридцяти

Невелике місто може дати 15–20 об'єктів. Тоді:

1. розширте перелік `tourism` (наприклад, додайте `theatre`, `place_of_worship`);
2. або зменште `N_LARGE` до того, що є;

і **обов'язково зазначте це у звіті** разом із числом.

---

3. Критерій «найкращі»

Ключовий крок роботи. Стовпця «цінність» у даних немає. Його треба сконструювати і захистити.

Сигнали, які є в OSM:

сигнал	що означає
`wikidata` / `wikipedia`	про об'єкт є стаття — хтось сторонній визнав його вартим опису
кількість колонок `name:*`	назву переклали N мовами — перекладають те, що шукають іноземці
`tourism=museum/gallery`	призначений для відвідування
`historic` заповнений	має історичну природу
площа полігона	розмір — **найслабший сигнал**: великим буває і склад

Завдання 3.1

Побудуйте числовий рейтинг. Вимоги:

- використати **щонайменше три** різні сигнали;
- звести різнорідні величини до спільної шкали (підказка: `rank(pct=True)`);
- **пояснити у звіті, чому саме такі ваги**.

In [12]:
```
metric = cand.to_crs(32635)
cand['area_m2'] = np.where(metric.geometry.geom_type.isin(['Polygon', 'Multi
                           metric.area.values, 0.0)

# стаття про сам об'єкт (subject:wikidata — це стаття про людину, не беремо)
cand['wiki']   = (cand['wikidata'].notna() | cand['wikipedia'].notna()).asty
cand['n_lang'] = cand[[c for c in cand.columns if c.startswith('name:')]].nc
# тип: музей/замок — 1, призначене для огляду — 0.6, інше історичне — 0.4, в
cand['type_s'] = np.select(
    [cand['tourism'].isin(['museum', 'gallery']) | cand['historic'].isin(['c
     cand['memorial'].isin(['statue', 'war_memorial']) | cand['historic'].eq
         | cand['tourism'].isin(['attraction', 'viewpoint']),
     cand['historic'].notna()],
    [1.0, 0.6, 0.4], default=0.2)
cand['score'] = (0.40 * cand['wiki']
                 + 0.25 * cand['n_lang'].rank(pct=True)
                 + 0.25 * cand['type_s']
                 + 0.10 * cand['area_m2'].rank(pct=True))

print('кандидатів   :', len(cand))
print('мають wiki   :', int(cand['wiki'].sum()))
```

```
кандидатів   : 218
мають wiki   : 28
```

Завдання 3.2 — обов'язкове порівняння

Побудуйте **другий** рейтинг за найпростішим критерієм — площею будівлі — і порахуйте, скільки об'єктів збігається в перших десяти.

Це головне питання роботи: обидва списки отримані з тих самих даних, тим самим кодом, без жодної помилки. Якщо вони різні — поясніть у звіті, що це означає для слова «найкращі».

In [13]:
```
N_SMALL, N_LARGE = 10, 30

top_score = cand.nlargest(N_LARGE, 'score').reset_index(drop=True)
top10     = top_score.head(N_SMALL)
top_naive = cand.nlargest(N_SMALL, 'area_m2').reset_index(drop=True)
overlap   = len(set(top10['name']) & set(top_naive['name']))
print('перетин: %d з %d' % (overlap, N_SMALL))
display(pd.DataFrame({'за вашим критерієм': list(top10['name']),
                      'за площею': list(top_naive['name'])}))
```

```
перетин: 2 з 10
```

	за вашим критерієм	за площею
0	Тернопільський замок	Тернопільський замок
1	Тернопільський краєзнавчий музей	Тернопільський краєзнавчий музей
2	Музей політичних в’язнів	Пам’ятний знак героям ОУН-УПА
3	Соломії Крушельницькій	Вічний вогонь
4	Степану Бандері	Військове кладовище
5	Іванові Франку	Місток
6	Горбачевському Івану	Св. Іван Непомук
7	Стецьку Ярославові	Місток
8	Королю Данилові Галицькому	Альтанка;Острів Кохання
9	Йосипу Сліпому	Танк т-34

Завдання 3.3 — дублікати

OSM наповнюють різні люди. Той самий музей трапляється двічі під різними назвами, а вежа буває окремим об'єктом усередині палацу. Знайдіть у `top_score` пари, що стоять ближче **60 метрів** одна до одної, і вирішіть, що з ними робити.

Рішення (лишити / злити / видалити) опишіть у звіті.

In [14]:
```
def close_pairs(gdf, limit_m=60):
    p = gdf.to_crs(32635).geometry.representative_point()
    XY = np.c_[p.x.values, p.y.values]
    D = np.sqrt(((XY[:, None, :] - XY[None, :, :]) ** 2).sum(-1))
    np.fill_diagonal(D, np.inf)
    out = [(i, j, D[i, j]) for i in range(len(XY)) for j in range(i + 1, ler
           if D[i, j] < limit_m]
    return sorted(out, key=lambda t: t[2])
```

```
for i, j, d in close_pairs(top_score):
    print('%5.1f м  %s  |  %s' % (d, top_score['name'].iloc[i], top_score['r
```

```
 13.8 м  Горбачевському Івану  |  Загиблим євреям
 28.3 м  Тристоронній годинник  |  Випадкова зустріч
 46.4 м  Тристоронній годинник  |  Незалежності
 51.7 м  Шептицькому Андрею  |  Пам’ятник жертвам депортації 1944–1946 років
з Лемківщини, Холмщини, Надсяння
 52.9 м  Іванові Франку  |  Картинна галерея
```

---

4. Старт: вокзал, і прив'язка точок до графа

In [15]:
```
print('станції у шарі:', len(stations))
print(stations[['name', 'railway', 'building']].dropna(how='all').head(12).t
```

```
станції у шарі: 6
                                    name  railway       building
0                                151 км     halt            NaN
1                    Тернопіль-Вантажний     halt            NaN
2                                Тернопіль  station            NaN
3            Автовокзал "Тернопіль" (АС-1)      NaN            NaN
4  Приміська автостанція "Тернопіль" (АС-2)      NaN            NaN
5                        Вокзал Тернопіль      NaN  train_station
```

In [16]:
```
station = stations[stations['building'].eq('train_station')].head(1)
print('старт:', station['name'].iloc[0])
```

```
старт: Вокзал Тернопіль
```

In [17]:
```
G = nx.DiGraph()
for t in nodes.itertuples():
    G.add_node(t.osmid, x=t.x, y=t.y)
for t in edges.itertuples():
    L = float(t.length)
    if G.has_edge(t.u, t.v):
        G[t.u][t.v]['length'] = min(G[t.u][t.v]['length'], L)
    else:
        G.add_edge(t.u, t.v, length=L)
print('граф: %d вузлів, %d ребер' % (G.number_of_nodes(), G.number_of_edges(

NODES_M = nodes.to_crs(32635)
NODE_XY = np.c_[NODES_M.geometry.x.values, NODES_M.geometry.y.values]
NODE_ID = nodes['osmid'].values

def snap(points_gdf):
    """Найближчий вузол графа до кожної точки. Рахуємо в метричній CRS."""
    pmet = points_gdf.to_crs(32635)
    ids, dist = [], []
    for p in pmet.geometry:
        d2 = (NODE_XY[:, 0] - p.x) ** 2 + (NODE_XY[:, 1] - p.y) ** 2
        k = int(np.argmin(d2))
        ids.append(NODE_ID[k]); dist.append(float(np.sqrt(d2[k])))
    return np.array(ids), np.array(dist)
```

```
def make_points(top):
    names = ['СТАРТ: ' + str(station['name'].iloc[0])] + list(top['name'])
    geoms = [station.geometry.iloc[0].representative_point()] + \
            [g.representative_point() for g in top.geometry]
    return gpd.GeoDataFrame({'name': names}, geometry=geoms, crs='EPSG:4326'
```

```
граф: 14314 вузлів, 38506 ребер
```

Завдання 4.1 — обов'язкова перевірка

Напишіть `check_snap`, яка повідомляє:

- скільки точок і скільки **різних** вузлів вони зайняли;
- медіану й максимум відстані «точка → вузол»;
- якщо вузлів менше, ніж точок — **назви точок, що злиплись**.

Контрольна ознака: на пішохідній мережі різних вузлів має бути **стільки ж, скільки точок**. Якщо менше — ви взяли автомобільну мережу або місто надто мале; зафіксуйте це у звіті, бо далі такі пари отримають нульову віддаль.

In [18]:
```
def check_snap(points, ids, dist, label):
    print('%s: точок %d, різних вузлів %d' % (label, len(ids), len(set(ids))
    print('відстань до вузла: медіана %.1f м, максимум %.1f м' % (np.median(
    if len(set(ids)) < len(ids):
        for nd in set(ids):
            names = [n for n, x in zip(points['name'], ids) if x == nd]
            if len(names) > 1:
                print('  злиплись:', ' | '.join(names))
```

---

5. Матриця віддалей

Дві матриці: **пряма лінія** і **дорогою**. Їхнє відношення — коефіцієнт обходу — показує, наскільки пряма лінія бреше.

Завдання 5.1

In [19]:
```
def road_matrix(ids):
    """Матриця найкоротших віддалей графом між заданими вузлами."""
    k = len(ids)
    D = np.full((k, k), np.inf)
    for i, s in enumerate(ids):
        lengths = nx.single_source_dijkstra_path_length(G, s, weight='length
        for j, t in enumerate(ids):
            D[i, j] = lengths.get(t, np.inf)
    np.fill_diagonal(D, 0.0)
    return D

def straight_matrix(points):
    p = points.to_crs(32635).geometry
```

```
    XY = np.c_[p.x.values, p.y.values]
    return np.sqrt(((XY[:, None, :] - XY[None, :, :]) ** 2).sum(-1))

def detour_factor(D, E):
    ok = np.isfinite(D) & (E > 0)
    return float(np.median(D[ok] / E[ok]))
```

Контрольні ознаки — звіртесь, не підганяючи:

- коефіцієнт обходу для пішохідної мережі у місті зазвичай **1,2–1,6**;
- якщо у вас рівно **1,0** — ви порівнюєте матрицю саму із собою;
- якщо **більше 2** — найімовірніше взято автомобільну мережу з односторонніми вулицями, або місто має річку/залізницю, яку нема де перетнути. Друге — цікавий результат, перше — помилка. Розберіться, що саме у вас, і напишіть це.

Якщо серед віддалей трапились `inf` — частина графа не зв'язана з рештою. Не замінюйте `inf` великим числом мовчки: спершу з'ясуйте, які це точки.

In [20]:
```
pts10 = make_points(top10)
ids10, dist10 = snap(pts10)
check_snap(pts10, ids10, dist10, 'мала задача')

t0 = time.time(); D10 = road_matrix(ids10); print('матриця за %.3f с' % (tim
E10 = straight_matrix(pts10)
print('недосяжних пар    :', int(np.isinf(D10).sum()))
print('асиметрія         : %.0f м' % float(np.max(np.abs(D10 - D10.T))))
print('коефіцієнт обходу : %.2f' % detour_factor(D10, E10))
```

```
мала задача: точок 11, різних вузлів 11
відстань до вузла: медіана 16.9 м, максимум 26.6 м
матриця за 0.163 с
недосяжних пар    : 0
асиметрія         : 0 м
коефіцієнт обходу : 1.29
```

> **Асиметрія.**
> D_{ij}
> $neq D_{ji}$
> — не помилка: односторонній рух робить шлях туди і назад різним. На пішохідній мережі асиметрія мала, на автомобільній — помітна.
> Подивіться на своє число і поясніть його у звіті.

---

6. Задача комівояжера

метод	що робить	складність
найближчий сусід	щоразу йде в найближчу невідвідану точку	$O(n^2)$

метод	що робить	складність
2-opt	розвертає ділянки готового маршруту, поки коротшає	$O(n^2)$ за прохід
Held–Karp	динамічне програмування за підмножинами — **точний**	$O(2^n n^2)$ часу, $O(2^n n)$ пам'яті

Held–Karp дано готовим: він не є метою роботи, але без нього ви не дізнаєтесь, **наскільки** помиляються перші два.

Завдання 6.1 — реалізуйте перші два методи

In [21]:
```python
def tour_length(D, tour):
    return float(sum(D[tour[i], tour[i + 1]] for i in range(len(tour) - 1)))

def nearest_neighbour(D):
    """Маршрут 0 -> ... -> 0. Щоразу йдемо в найближчу невідвідану точку."""
    tour, left = [0], set(range(1, len(D)))
    while left:
        nxt = min(left, key=lambda j: D[tour[-1], j])
        tour.append(nxt); left.remove(nxt)
    return tour + [0]

def two_opt(D, tour):
    """Беремо маршрут і розвертаємо ділянки, поки довжина зменшується."""
    best, L_best = list(tour), tour_length(D, tour)
    improved = True
    while improved:
        improved = False
        for i in range(1, len(best) - 2):
            for j in range(i + 1, len(best) - 1):
                new = best[:i] + best[i:j + 1][::-1] + best[j + 1:]
                L = tour_length(D, new)
                if L < L_best - 1e-9:
                    best, L_best, improved = new, L, True
    return best
```

In [22]:
```python
def held_karp(D):
    """Точний розв'язок. Вузол 0 — старт і фініш. ДАНО, писати не треба."""
    k = len(D); N = k - 1; INF = float('inf')
    dp = [[INF] * N for _ in range(1 << N)]
    par = [[-1] * N for _ in range(1 << N)]
    for j in range(N):
        dp[1 << j][j] = D[0, j + 1]
    for S in range(1 << N):
        for j in range(N):
            cur = dp[S][j]
            if cur == INF or not (S >> j) & 1:
                continue
            for t in range(N):
                if (S >> t) & 1:
                    continue
                S2 = S | (1 << t); v = cur + D[j + 1, t + 1]
```

```
                if v < dp[S2][t]:
                    dp[S2][t] = v; par[S2][t] = j
    full = (1 << N) - 1
    bj = min(range(N), key=lambda j: dp[full][j] + D[j + 1, 0])
    best = dp[full][bj] + D[bj + 1, 0]
    path = []; S = full; j = bj
    while j != -1:
        path.append(j + 1); pj = par[S][j]; S ^= (1 << j); j = pj
    return [0] + path[::-1] + [0], float(best)
```

Самоперевірка

Ці твердження мають виконатись для **будь-якого** міста. Якщо хоч одне впало — шукайте помилку в коді, а не в даних.

In [23]:
```
def check_tour(tour, k, label):
    assert tour[0] == 0 and tour[-1] == 0, '%s: маршрут має починатись і зак
    assert sorted(tour[:-1]) == list(range(k)), '%s: кожна точка рівно один
    print('%s: структура маршруту правильна (%d точок)' % (label, k))

nn10 = nearest_neighbour(D10)
op10 = two_opt(D10, nn10)
ex10, L_opt = held_karp(D10)

for t, lbl in [(nn10, 'найближчий сусід'), (op10, '2-opt'), (ex10, 'Held-Kar
    check_tour(t, len(ids10), lbl)

assert tour_length(D10, op10) <= tour_length(D10, nn10) + 1e-6, '2-opt не мо
assert L_opt <= tour_length(D10, op10) + 1e-6, 'точний розв\u2019язок не мож
print()
print('усі перевірки пройдено')
```

```
найближчий сусід: структура маршруту правильна (11 точок)
2-opt: структура маршруту правильна (11 точок)
Held-Karp: структура маршруту правильна (11 точок)

усі перевірки пройдено
```

In [24]:
```
rows = []
for name, tour in [('найближчий сусід', nn10), ('2-opt', op10), ('Held-Karp
    L = tour_length(D10, tour)
    rows.append({'метод': name, 'довжина, м': round(L),
                 'гірше оптимуму, %': round(100 * (L / L_opt - 1), 2)})
display(pd.DataFrame(rows).set_index('метод'))

print()
print('оптимальний порядок:')
for step, i in enumerate(ex10):
    print('  %2d. %s' % (step, pts10['name'].iloc[i]))
print()
print('довжина: %.2f км' % (L_opt / 1000))
```

метод	довжина, м	гірше оптимуму, %
найближчий сусід	4936	22.24
2-opt	4065	0.68
Held-Karp (точний)	4038	0.00

```
оптимальний порядок:
   0. СТАРТ: Вокзал Тернопіль
   1. Стецьку Ярославові
   2. Степану Бандері
   3. Тернопільський замок
   4. Горбачевському Івану
   5. Королю Данилові Галицькому
   6. Йосипу Сліпому
   7. Іванові Франку
   8. Тернопільський краєзнавчий музей
   9. Соломії Крушельницькій
  10. Музей політичних в'язнів
  11. СТАРТ: Вокзал Тернопіль

довжина: 4.04 км
```

Завдання 6.2 — велика задача

Спершу оцініть, чи можливий тут точний розв'язок. Порахуйте для $n = 11, 16, 21, 26, 31$ розмір таблиці Held–Karp ($2^{n-1} cdot (n-1)$ клітин) і скільки це гігабайтів при 8 байтах на клітину.

In [25]:
```
growth = pd.DataFrame({'n': [11, 16, 21, 26, 31]})
growth['клітин'] = 2 ** (growth['n'] - 1) * (growth['n'] - 1)
growth['ГБ'] = (growth['клітин'] * 8 / 1e9).round(3)
display(growth.set_index('n'))
```

n	клітин	ГБ
11	10240	0.000
16	491520	0.004
21	20971520	0.168
26	838860800	6.711
31	32212254720	257.698

Подивіться на останній рядок і напишіть у звіті, **чому саме** точний розв'язок на 30 точках недоступний. Слово «довго» відповіддю не є.

In [26]:
```
pts30 = make_points(top_score)
ids30, dist30 = snap(pts30)
check_snap(pts30, ids30, dist30, 'велика задача')

D30 = road_matrix(ids30)
E30 = straight_matrix(pts30)
print('коефіцієнт обходу: %.2f' % detour_factor(D30, E30))

nn30 = nearest_neighbour(D30)
t0 = time.time(); op30 = two_opt(D30, nn30); t_op30 = time.time() - t0
check_tour(op30, len(ids30), '2-opt на 30 точках')

L_nn30, L_op30 = tour_length(D30, nn30), tour_length(D30, op30)
print()
print('найближчий сусід : %8.0f м' % L_nn30)
print('2-opt            : %8.0f м  за %.3f с  (коротше на %.1f%%)'
      % (L_op30, t_op30, 100 * (1 - L_op30 / L_nn30)))
print()
print('ЦЕ — база порівняння для лабораторної роботи № 8.')
```

```
велика задача: точок 31, різних вузлів 31
відстань до вузла: медіана 13.4 м, максимум 35.4 м
коефіцієнт обходу: 1.23
2-opt на 30 точках: структура маршруту правильна (31 точок)

найближчий сусід :    20914 м
2-opt            :    19752 м  за 0.005 с  (коротше на 5.6%)

ЦЕ — база порівняння для лабораторної роботи № 8.
```

---

7. Просторова індексація: H3

H3 — ієрархічна сітка шестикутників. Рівень 9 — приблизно 200 м по ребру.

📖 https://h3geo.org/docs/core-library/restable/

Завдання 7.1

Розкладіть **усіх** кандидатів по комірках H3 рівня 9 і побудуйте картограму щільності. На тій самій карті покажіть межу міста і точки вашого маршруту.

In [27]:
```
import h3
from shapely.geometry import Polygon

H3_RES = 9
print('середнє ребро комірки: %.0f м' % h3.average_hexagon_edge_length(H3_RE

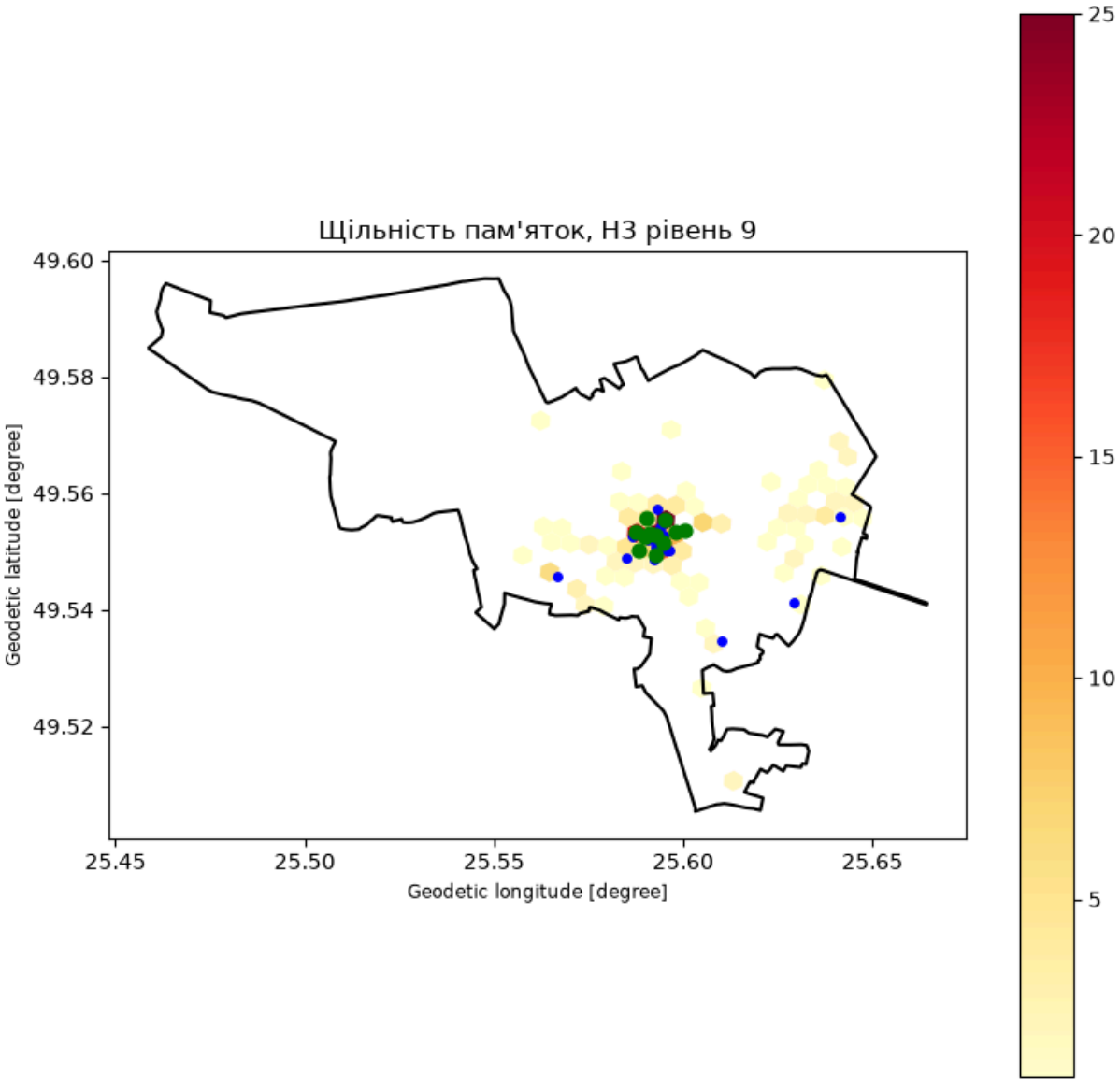
cand['h3'] = [h3.latlng_to_cell(p.y, p.x, H3_RES) for p in cand.geometry.rep
dens = cand['h3'].value_counts()
hexes = gpd.GeoDataFrame({'n': dens.values},
```

```
                          geometry=[Polygon([(lng, lat) for lat, lng in h3.ce
                                   for c in dens.index], crs='EPSG:4326')

half = int((dens.cumsum() < dens.sum() / 2).sum()) + 1
print('комірок з пам\'ятками: %d, половина пам\'яток у %d з них' % (len(dens

ax = hexes.plot(column='n', cmap='YlOrRd', legend=True, figsize=(9, 9))
boundary.boundary.plot(ax=ax, color='black')
pts30.plot(ax=ax, color='blue', markersize=15)
pts10.plot(ax=ax, color='green', markersize=40)
ax.set_title('Щільність пам\'яток, H3 рівень %d' % H3_RES)
plt.show()
```

```
середнє ребро комірки: 201 м
комірок з пам'ятками: 70, половина пам'яток у 8 з них
```



> **Пастка.** `h3.cell_to_boundary` повертає пари **(lat, lng)**, а `shapely.Polygon` чекає **(x, y)**, тобто (lng, lat). Переплутаєте — місто опиниться в Індійському океані. Це видно одразу, тому дивіться на карту.

Питання до звіту

У скількох комірках зосереджено половину всіх пам'яток вашого міста? Що це говорить про форму маршруту?

---

8. Просторове поєднання

Завдання 8.1

Для кожної точки маршруту знайдіть найближчий заклад із шару `amenities` (`sjoin_nearest`) і виведіть таблицю з відстанню в метрах.

Нагадування: `sjoin_nearest` рахує в одиницях CRS. У градусах результат буде безглуздим — перепроєктуйте обидва шари в метричну систему.

In [28]:
```
near = gpd.sjoin_nearest(pts30.to_crs(32635),
                         amenities.to_crs(32635)[['amenity', 'name', 'geomet
                         how='left', distance_col='відстань_м', lsuffix='точ
near = near[~near.index.duplicated()]
display(near[['name_точка', 'amenity', 'name_заклад', 'відстань_м']].round(0
```

	name_точка	amenity	name_заклад	відстань_м
0	СТАРТ: Вокзал Тернопіль	toilets	NaN	9.0
1	Тернопільський замок	restaurant	Веранда	30.0
2	Тернопільський краєзнавчий музей	restaurant	Дружня ресторація „Мамонт“	41.0
3	Музей політичних в’язнів	cafe	Tikithai	40.0
4	Соломії Крушельницькій	restaurant	Хінкальня	58.0
5	Степану Бандері	toilets	NaN	137.0
6	Іванові Франку	cafe	NaN	35.0
7	Горбачевському Івану	cafe	Eggo Coffee	65.0
8	Стецьку Ярославові	cafe	M'ясо	23.0
9	Королю Данилові Галицькому	pharmacy	NaN	94.0
10	Йосипу Сліпому	drinking_water	NaN	37.0
11	Гнатюку Володимиру	restaurant	Neo Room	194.0
12	Тарасу Шевченко	cafe	Parma	59.0
13	Тристоронній годинник	restaurant	Клуб файного міста	41.0
14	Ігорю Гереті	cafe	Big Cup	14.0
15	Скульптура Ангела-хоронителя	restaurant	Ковчег	105.0
16	Шептицькому Андрею	pharmacy	Рецептіка	83.0
17	Випадкова зустріч	restaurant	Пасаж Адлера 13	36.0
18	Незалежності	restaurant	Файне місто	43.0
19	Пам’ятник жертвам депортації 1944-1946 років з...	pharmacy	Рецептіка	32.0
20	Небесній сотні	restaurant	Дружня ресторація „Мамонт“	54.0
21	Митрополиту Липківському	pharmacy	Здорова родина	34.0
22	Загиблим євреям	cafe	Eggo Coffee	52.0
23	Симону Петлюрі	restaurant	NaN	49.0
24	Погруддя Богдана Лепкого	cafe	Авокадо	209.0
25	Меморіал українських січових стрільців	cafe	Колиба	497.0
26	Леоніду Ковальчуку	restaurant	Майдан	33.0
27	Дерево Щастя	restaurant	Веранда	96.0
28	Пара лелек	cafe	Ляпота	43.0

	name_точка	amenity	name_заклад	відстань_м
29	Обласний художній музей	cafe	Шеф Панда	109.0
30	Картинна галерея	cafe	Карма Кава	14.0

---

9. Маршрут на карті

Лінія маршруту — це **геометрія ребер графа**, а не відрізки між пам'ятками.

Завдання 9.1

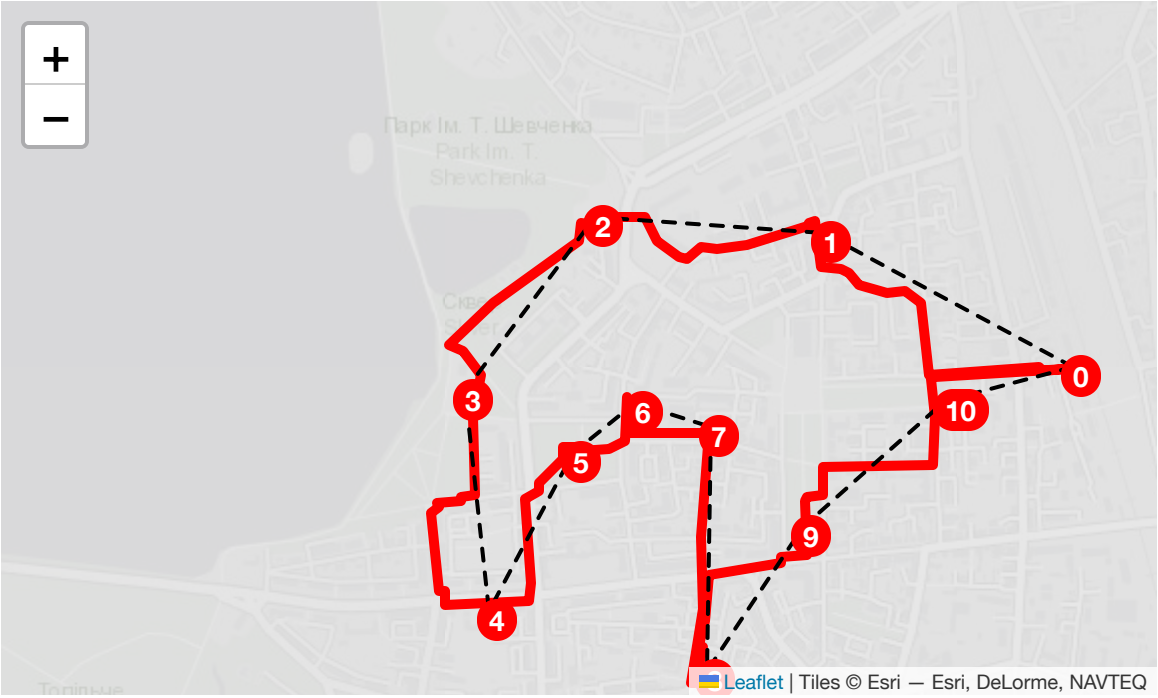
Побудуйте карту, на якій **обидві** лінії: справжній маршрут по ребрах і прямі відрізки між точками. Різниця має бути видно оком.

In [29]:
```
node_xy = nodes.set_index('osmid')[['x', 'y']]

def route_coords(tour, ids):
    """Координати маршруту по ребрах графа у форматі (lat, lon) для folium."
    coords = []
    for a, b in zip(tour[:-1], tour[1:]):
        try:
            path = nx.shortest_path(G, ids[a], ids[b], weight='length')
        except nx.NetworkXNoPath:
            path = [ids[a], ids[b]]
        for nd in path:
            coords.append((float(node_xy.loc[nd, 'y']), float(node_xy.loc[nc
    return coords

c = pts10.geometry.union_all().centroid
fmap = folium.Map(location=[c.y, c.x], zoom_start=15, tiles=BASEMAP)
folium.PolyLine(route_coords(ex10, ids10), color='red', weight=5,
                tooltip='по ребрах графа').add_to(fmap)
folium.PolyLine([(pts10.geometry.y.iloc[i], pts10.geometry.x.iloc[i]) for i
                color='black', weight=2, dash_array='6', tooltip='прямі відр
for step, i in enumerate(ex10[:-1]):
    p = pts10.geometry.iloc[i]
    folium.Marker([p.y, p.x], tooltip=pts10['name'].iloc[i],
                  icon=folium.DivIcon(html='<b style="font-size:14px;color:w
                                             'padding:2px 6px;border-radius:10
                  ).add_to(fmap)
fmap.save('lr5_маршрут.html')
fmap
```

Out[29]:



---

10. Результати для лабораторної роботи № 8

Збережіть файл, який ЛР8 прочитає і спробує побити.

In [30]:

```
result = {
    'студент': STUDENT_NAME,
    'номер': LIST_NUMBER,
    'місто': CITY,
    'мала_задача': {
        'точок': len(ids10),
        'оптимум_м': round(L_opt, 1),
        'оптимальний_порядок': [str(pts10['name'].iloc[i]) for i in ex10],
        'найближчий_сусід_м': round(tour_length(D10, nn10), 1),
        'двоопт_м': round(tour_length(D10, op10), 1),
    },
    'велика_задача': {
        'точок': len(ids30),
        'найближчий_сусід_м': round(L_nn30, 1),
        'двоопт_м': round(L_op30, 1),
        'двоопт_с': round(t_op30, 4),
        'точки': [{'name': str(pts30['name'].iloc[i]),
                   'lat': float(pts30.geometry.y.iloc[i]),
                   'lon': float(pts30.geometry.x.iloc[i])} for i in range(le
        'матриця_віддалей_м': [[None if not np.isfinite(v) else round(float(
                               for v in row] for row in D30],
    },
}
with open('lr5_результати.json', 'w', encoding='utf-8') as f:
    json.dump(result, f, ensure_ascii=False, indent=1)
print('збережено lr5_результати.json (%.1f КБ)' % (os.path.getsize('lr5_резу
```

```
збережено lr5_результати.json (15.3 КБ)
```

---

11. Звіт

Відповіді пишіть просто в наступній комірці, числами зі своїх обчислень.

1. **Скільки об'єктів ви відкинули** на кроці 2 і за яким правилом? Чи погодились би ви віддати цей фільтр замовнику без пояснень?
2. **Які сигнали ввійшли у ваш рейтинг і з якими вагами?** Чому саме так?
3. **Скільки об'єктів збіглося** у ваших двох рейтингах із десяти? Що це означає для слова «найкращі»?
4. **Який у вас коефіцієнт обходу** і чи типовий він? Якщо ні — що в місті його пояснює?
5. **На скільки відсотків** найближчий сусід гірший за точний розв'язок? Чи здивувало вас це число?
6. **Чому точний розв'язок на 30 точках недоступний?** Назвіть конкретну величину, яка цьому заважає.
7. **Що у ваших даних виявилось неправильним** — дублікати, пропущені назви, об'єкти не на місці? Як ви з цим вчинили?

Декларація про використання штучного інтелекту

Зазначте, чи користувались ШІ-помічником, у яких саме місцях і як перевіряли отриманий код. Використання дозволене; **неперевірене** використання — ні.

In [31]:
```
REPORT = '''
1. У шарі heritage 266 об'єктів, і всі мають теги historic=* або tourism із
   Відкинуто 48 об'єктів без name: такий об'єкт неможливо показати туристу і
   перевірити, що це. Лишилось 218 кандидатів (205 Point, 13 Polygon). Без г
   замовнику я б фільтр не віддав: він бере лише теги historic/tourism, тому
   без них у вибірку не потрапляють, а серед кандидатів лишаються меморіальн
   (historic=memorial).

2. score = 0.40·wiki + 0.25·rank(n_lang) + 0.25·тип + 0.10·rank(площа), rank
   * wiki (є wikidata/wikipedia самого об'єкта) — найсильніший сигнал: незал
     яке пройшло модерацію Вікіпедії. subject:wikidata не враховую, бо це ст
   * n_lang (кількість мов назви) — інтерес іноземців.
   * тип: музей/замок = 1, статуя/атракція = 0.6, інше історичне = 0.4, скул
     застосунок веде туди, куди ходять.
   * площа — найслабший сигнал (великим буває і склад), у точок вона 0.
   wiki мають 28 кандидатів з 218.

3. Збіглися 2 з 10: Тернопільський замок і краєзнавчий музей. За площею в то
   військове кладовище, два «Містки» в парку, альтанка і танк Т-34. Обидва с
   з тих самих даних без помилки, отже «найкращі» — це не властивість даних,
   аналітика, яке треба записати й обґрунтувати.
```

```
4. Коефіцієнт обходу 1.29 для 10 точок і 1.23 для 30 — типово для пішохідної
   (1.2–1.6). Асиметрія 0 м: пішохідний граф osmnx двонапрямлений, односторо
   пішохода не діє.

5. Найближчий сусід — 4936 м проти оптимуму 4038 м, тобто на 22.24 % гірше.
   лише на 0.68 % гірше. Здивувало, що жадібний метод помиляється на п'яту ч
   на 10 точках, а 2-opt прибирає майже всю помилку. На 30 точках 2-opt скор
   з 20914 до 19752 м (на 5.6 %) за 0.005 с.

6. Заважає пам'ять. Таблиця Held–Karp для n = 31 має 2^30·30 = 32 212 254 72
   257.7 ГБ при 8 байтах на клітину. Це в десятки разів більше за оперативну

7. 48 об'єктів без назви; дві назви в одному полі («Альтанка;Острів Кохання»
   площею двічі «Місток» — одна споруда двома полігонами. У топ-30 знайдено
   (наприклад, Тристоронній годинник — «Випадкова зустріч», 28 м; пам'ятник
   Картинна галерея, 53 м). Я перевірив кожну пару за назвою: це різні об'єк
   Злипання немає: 31 точка сіла на 31 різний вузол.
   Живий запит із ноутбука не пройшов: Overpass відповідав 502 (сервер перев
   Тому знімок зібрано скриптом викладача fetch_city_osm.py. За його manifes
   запит тривав 416 с і передав 7.06 МБ. Читання знімка займає ~0.5 с, тобто
   швидше, і передає 0 МБ.

Висновок, якого не було в завданні: сигнал wiki перетворює «найвартісніші па
«пам'ятники відомим людям»: 7 з 10 у моєму топі — статуї й погруддя, музеїв
Отже, рейтинг відображає те, яким об'єктам мапери проставляють wikidata, не
ніж цінність самих об'єктів.

Декларація про використання ШІ:
Код і текст звіту підготовлено з допомогою ШІ-помічника (Claude Code). Перев
вбудовані самоперевірки ноутбука (check_tour, нерівності 2-opt ≤ NN і Held-K
check_snap, звірка коефіцієнта обходу з контрольним діапазоном, перегляд пар
'''
print(REPORT)
```

1. У шарі heritage 266 об'єктів, і всі мають теги historic=* або tourism із TOURISM_OK.
   Відкинуто 48 об'єктів без name: такий об'єкт неможливо показати туристу і не
   перевірити, що це. Лишилось 218 кандидатів (205 Point, 13 Polygon). Без пояснень
   замовнику я б фільтр не віддав: він бере лише теги historic/tourism, тому об'єкти
   без них у вибірку не потрапляють, а серед кандидатів лишаються меморіальні дошки
   (historic=memorial).

2. score = 0.40·wiki + 0.25·rank(n_lang) + 0.25·тип + 0.10·rank(площа), rank = rank(pct=True).
   * wiki (є wikidata/wikipedia самого об'єкта) — найсильніший сигнал: незалежне визнання,
     яке пройшло модерацію Вікіпедії. subject:wikidata не враховую, бо це стаття про людину.
   * n_lang (кількість мов назви) — інтерес іноземців.
   * тип: музей/замок = 1, статуя/атракція = 0.6, інше історичне = 0.4, скульптура = 0.2:
     застосунок веде туди, куди ходять.
   * площа — найслабший сигнал (великим буває і склад), у точок вона 0.
   wiki мають 28 кандидатів з 218.

3. Збіглися 2 з 10: Тернопільський замок і краєзнавчий музей. За площею в топ потрапили
   військове кладовище, два «Містки» в парку, альтанка і танк Т-34. Обидва списки побудовано
   з тих самих даних без помилки, отже «найкращі» — це не властивість даних, а рішення
   аналітика, яке треба записати й обґрунтувати.

4. Коефіцієнт обходу 1.29 для 10 точок і 1.23 для 30 — типово для пішохідної мережі
   (1.2–1.6). Асиметрія 0 м: пішохідний граф osmnx двонапрямлений, односторонній рух на
   пішохода не діє.

5. Найближчий сусід — 4936 м проти оптимуму 4038 м, тобто на 22.24 % гірше. 2-opt — 4065 м,
   лише на 0.68 % гірше. Здивувало, що жадібний метод помиляється на п'яту частину навіть
   на 10 точках, а 2-opt прибирає майже всю помилку. На 30 точках 2-opt скоротив маршрут
   з 20914 до 19752 м (на 5.6 %) за 0.005 с.

6. Заважає пам'ять. Таблиця Held–Karp для n = 31 має 2^30·30 = 32 212 254 720 клітин, тобто
   257.7 ГБ при 8 байтах на клітину. Це в десятки разів більше за оперативну пам'ять ноутбука.

7. 48 об'єктів без назви; дві назви в одному полі («Альтанка;Острів Кохання»); у топ-10 за
   площею двічі «Місток» — одна споруда двома полігонами. У топ-30 знайдено 5 пар ближче 60 м

(наприклад, Тристоронній годинник — «Випадкова зустріч», 28 м; пам'ятник Франку —
Картинна галерея, 53 м). Я перевірив кожну пару за назвою: це різні об'єкти, тож лишаю всі.
Злипання немає: 31 точка сіла на 31 різний вузол.
Живий запит із ноутбука не пройшов: Overpass відповідав 502 (сервер перевантажений).
Тому знімок зібрано скриптом викладача fetch_city_osm.py. За його manifest.json живий
запит тривав 416 с і передав 7.06 МБ. Читання знімка займає ~0.5 с, тобто в ~800 разів
швидше, і передає 0 МБ.

Висновок, якого не було в завданні: сигнал wiki перетворює «найвартісніші пам'ятки» на
«пам'ятники відомим людям»: 7 з 10 у моєму топі — статуї й погруддя, музеїв лише 2.
Отже, рейтинг відображає те, яким об'єктам мапери проставляють wikidata, не менше,
ніж цінність самих об'єктів.

Декларація про використання ШІ:
Код і текст звіту підготовлено з допомогою ШІ-помічника (Claude Code). Перевірка:
вбудовані самоперевірки ноутбука (check_tour, нерівності 2-opt ≤ NN і Held-Karp ≤ 2-opt),
check_snap, звірка коефіцієнта обходу з контрольним діапазоном, перегляд пар-дублікатів і карти.